

循序渐进，学习开发 xv6-riscv

第 2 章 启动 (Boot)

汪辰



目录

01

SoC 的概念

02

QEMU-virt 系统引导过程

03

xv6 的启动实现

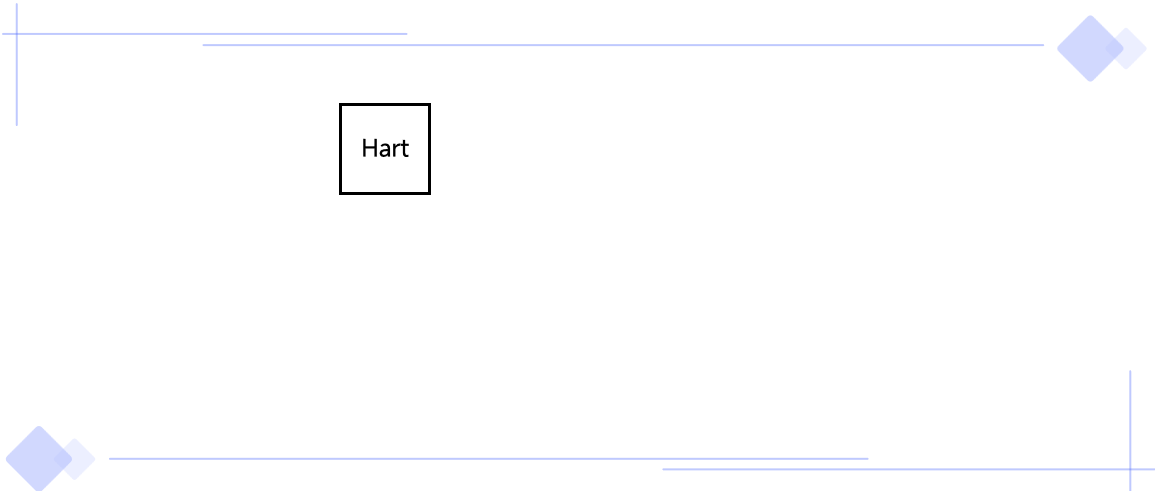


01

SoC 的概念



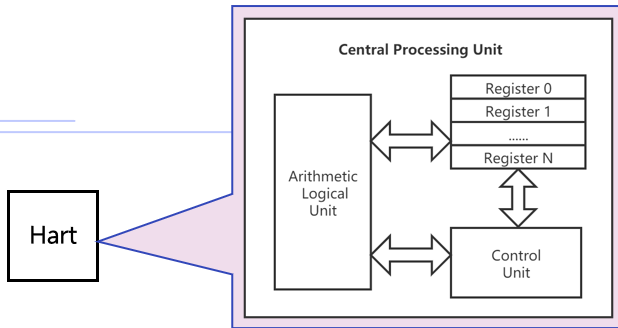
什么是 SoC



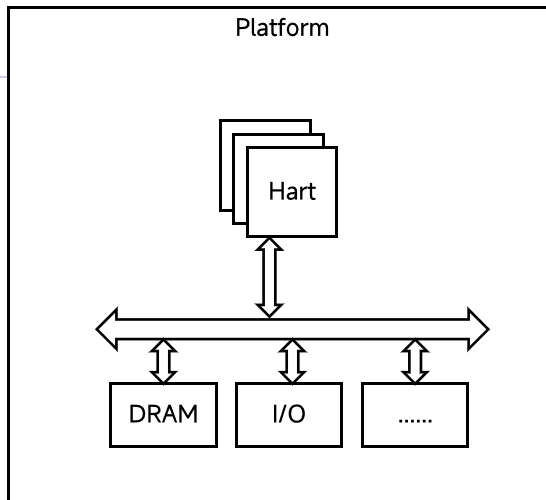
A diagram illustrating the layout of a System on a Chip (SoC). It features a central square block labeled "Hart". This block is positioned between two horizontal lines. The top horizontal line starts from the left edge and ends with a blue diamond shape. The bottom horizontal line starts with a blue diamond shape and ends at the right edge. There are also vertical lines at the left and right ends, forming a frame around the central "Hart" block.

Hart

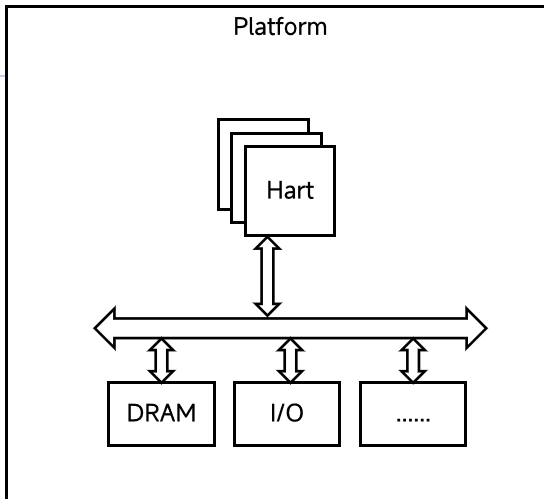
什么是 SoC



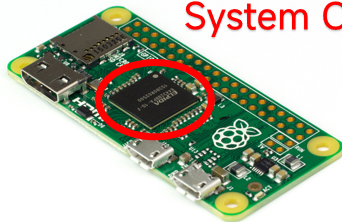
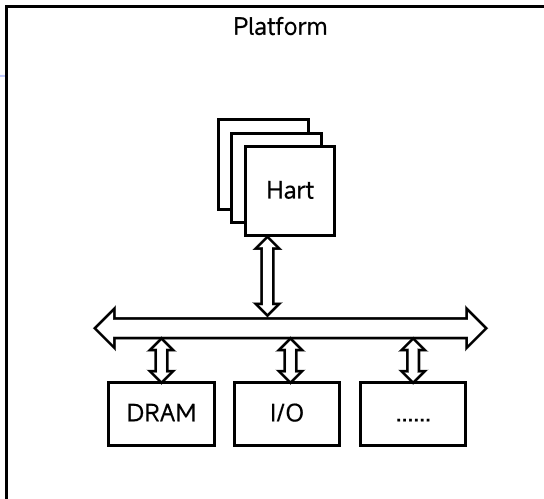
什么是 SoC



什么是 SoC



什么是 SoC



System On Chip

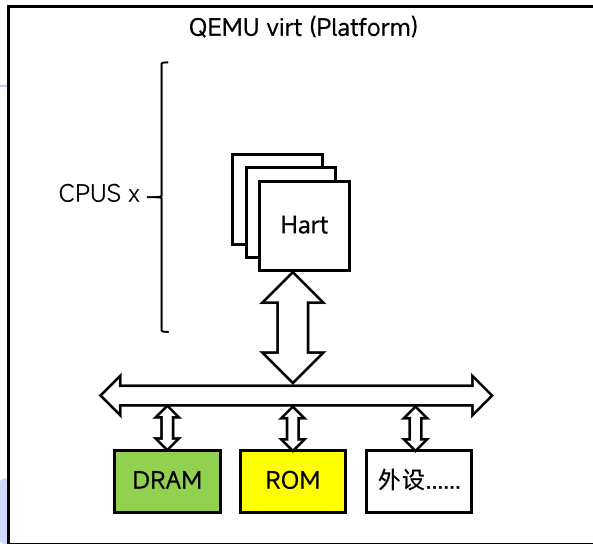


02

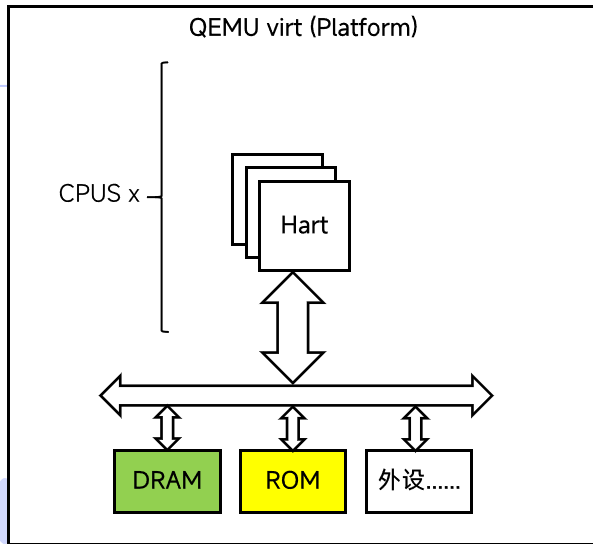
QEMU-virt 系统 引导过程



硬件地址空间



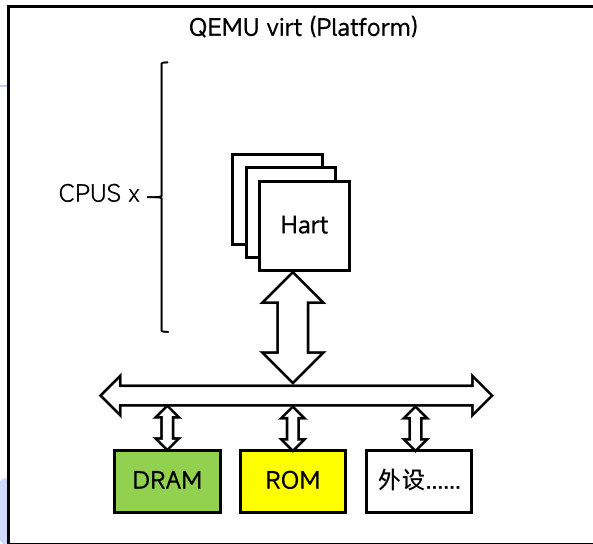
硬件地址空间



硬件编址的方式：

- 端口映射 I/O（独立编址）：内存地址空间和其他外设 I/O 地址空间独立。
- 内存映射 I/O（统一编址）：内存地址空间和其他外设 I/O 地址空间纳入统一的地址空间。

硬件地址空间



硬件编址的方式：

- 端口映射 I/O（独立编址）：内存地址空间和其他外设 I/O 地址空间独立。
- 内存映射 I/O（统一编址）：内存地址空间和其他外设 I/O 地址空间纳入统一的地址空间。

硬件地址空间

<https://github.com/qemu/qemu/blob/master/hw/riscv/virt.c>

```
static const MemMapEntry virt_memmap[] = {
    [VIRT_DEBUG] = { 0x0, 0x100 },
    [VIRT_MROM] = { 0x1000, 0xf000 },
    [VIRT_TEST] = { 0x100000, 0x1000 },
    [VIRT_RTC] = { 0x101000, 0x1000 },
    [VIRT_CLINT] = { 0x2000000, 0x10000 },
    [VIRT_ACLINT_SSWI] = { 0x2f00000, 0x4000 },
    [VIRT_PCIE_PIO] = { 0x3000000, 0x10000 },
    [VIRT_IOMMU_SYS] = { 0x3010000, 0x1000 },
    [VIRT_PLATFORM_BUS] = { 0x4000000, 0x2000000 },
    [VIRT_PLIC] = { 0xc000000, VIRT_PLIC_SIZE(VIRT_CPUS_MAX * 2) },
    [VIRT_APLIC_M] = { 0xc000000, APLIC_SIZE(VIRT_CPUS_MAX) },
    [VIRT_APLIC_S] = { 0xd000000, APLIC_SIZE(VIRT_CPUS_MAX) },
    [VIRT_UART0] = { 0x10000000, 0x100 },
    [VIRT_VIRTIO] = { 0x10001000, 0x1000 },
    [VIRT_FW_CFG] = { 0x10100000, 0x18 },
    [VIRT_FLASH] = { 0x20000000, 0x4000000 },
    [VIRT_IMSIC_M] = { 0x24000000, VIRT_IMSIC_MAX_SIZE },
    [VIRT_IMSIC_S] = { 0x28000000, VIRT_IMSIC_MAX_SIZE },
    [VIRT_PCIE_ECAM] = { 0x30000000, 0x10000000 },
    [VIRT_PCIE_MMIO] = { 0x40000000, 0x40000000 },
    [VIRT_DRAM] = { 0x80000000, 0x0 },
};
```

硬件地址空间

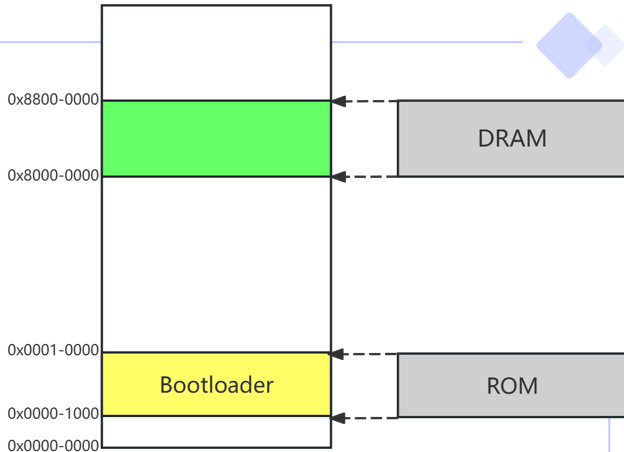
<https://github.com/qemu/qemu/blob/master/hw/riscv/virt.c>

```
static const MemMapEntry virt_memmap[] = {  
    [VIRT_DEBUG] = { 0x0, 0x100 },  
    [VIRT_MROM] = { 0x1000, 0xf000 },  
    [VIRT_FEST] = { 0x100000, 0x1000 },  
    [VIRT_RTC] = { 0x101000, 0x1000 },  
    [VIRT_CLINT] = { 0x200000, 0x1000 },  
    [VIRT_ACLINT_SSWI] = { 0x2f0000, 0x4000 },  
    [VIRT_PCIE_PIO] = { 0x300000, 0x1000 },  
    [VIRT_IOMMU_SYS] = { 0x301000, 0x1000 },  
    [VIRT_PLATFORM_BUS] = { 0x400000, 0x200000 },  
    [VIRT_PLIC] = { 0xc00000, VIRT_PLIC_SIZE(VIRT_CPUS_MAX * 2) },  
    [VIRT_APLIC_M] = { 0xc00000, APLIC_SIZE(VIRT_CPUS_MAX) },  
    [VIRT_APLIC_S] = { 0xd00000, APLIC_SIZE(VIRT_CPUS_MAX) },  
    [VIRT_UART0] = { 0x1000000, 0x100 },  
    [VIRT_VIRTIO] = { 0x1000100, 0x1000 },  
    [VIRT_FW_CFG] = { 0x1010000, 0x18 },  
    [VIRT_FLASH] = { 0x2000000, 0x400000 },  
    [VIRT_IMSIC_M] = { 0x2400000, VIRT_IMSIC_MAX_SIZE },  
    [VIRT_IMSIC_S] = { 0x2800000, VIRT_IMSIC_MAX_SIZE },  
    [VIRT_PCIE_ECAM] = { 0x3000000, 0x1000000 },  
    [VIRT_PCIE_MMIO] = { 0x4000000, 0x4000000 },  
    [VIRT_DRAM] = { 0x8000000, 0x0 },  
};
```

硬件地址空间

<https://github.com/qemu/qemu/blob/master/hw/riscv/virt.c>

```
static const MemMapEntry virt_memmap[] = {  
    [VIRT_DEBUG] = { 0x0, 0x100 },  
    [VIRT_MROM] = { 0x1000, 0xf000 },  
    [VIRT_FEST] = { 0x100000, 0x1000 },  
    [VIRT_RTC] = { 0x101000, 0x1000 },  
    [VIRT_CLINT] = { 0x200000, 0x1000 },  
    [VIRT_ACLINT_SSWI] = { 0x2f0000, 0x4000 },  
    [VIRT_PCIE_PIO] = { 0x300000, 0x10000 },  
    [VIRT_IOMMU_SYS] = { 0x301000, 0x1000 },  
    [VIRT_PLATFORM_BUS] = { 0x400000, 0x2000000 },  
    [VIRT_PLIC] = { 0xc00000, VIRT_PLIC_SIZE(VIRT_CPUS_MAX * 2) },  
    [VIRT_APLIC_M] = { 0xc00000, APLIC_SIZE(VIRT_CPUS_MAX) },  
    [VIRT_APLIC_S] = { 0xd00000, APLIC_SIZE(VIRT_CPUS_MAX) },  
    [VIRT_UART0] = { 0x1000000, 0x100 },  
    [VIRT_VIRTIO] = { 0x1000100, 0x1000 },  
    [VIRT_FW_CFG] = { 0x1010000, 0x18 },  
    [VIRT_FLASH] = { 0x2000000, 0x4000000 },  
    [VIRT_IMSIC_M] = { 0x2400000, VIRT_IMSIC_MAX_SIZE },  
    [VIRT_IMSIC_S] = { 0x2800000, VIRT_IMSIC_MAX_SIZE },  
    [VIRT_PCIE_ECAM] = { 0x3000000, 0x1000000 },  
    [VIRT_PCIE_MMIO] = { 0x4000000, 0x4000000 },  
    [VIRT_DRAM] = { 0x8000000, 0x0 },  
};
```



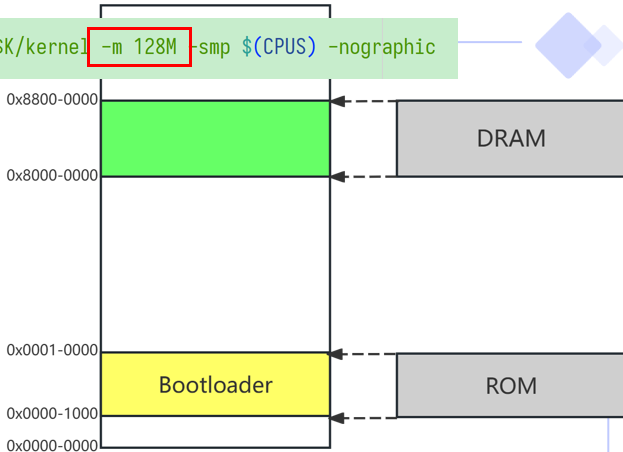
硬件地址空间

<https://github.com/qemu/qemu/blob/master/hw/riscv/virt.c>

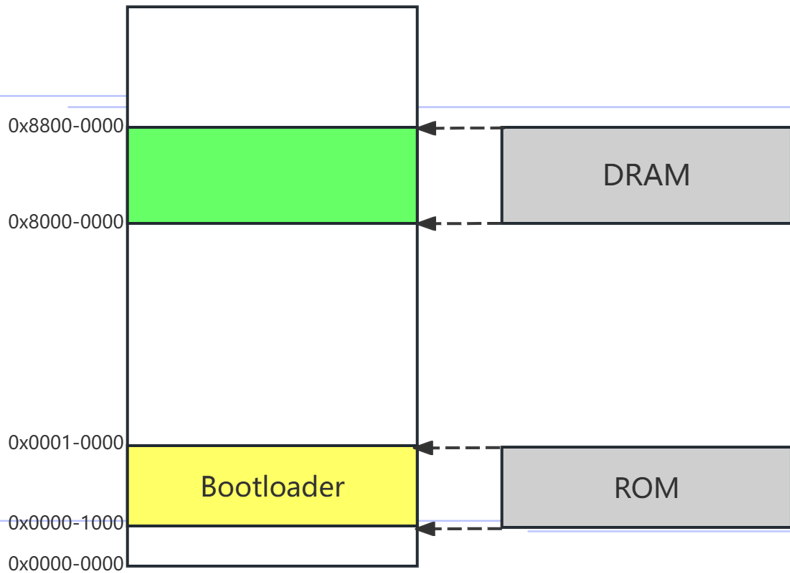
```
static const MemMapEntry virt_memmap[] = {
```

```
QEMUOPTS = -machine virt -bios none -kernel $K/kernel -m 128M -smp $(CPUS) -nographic
```

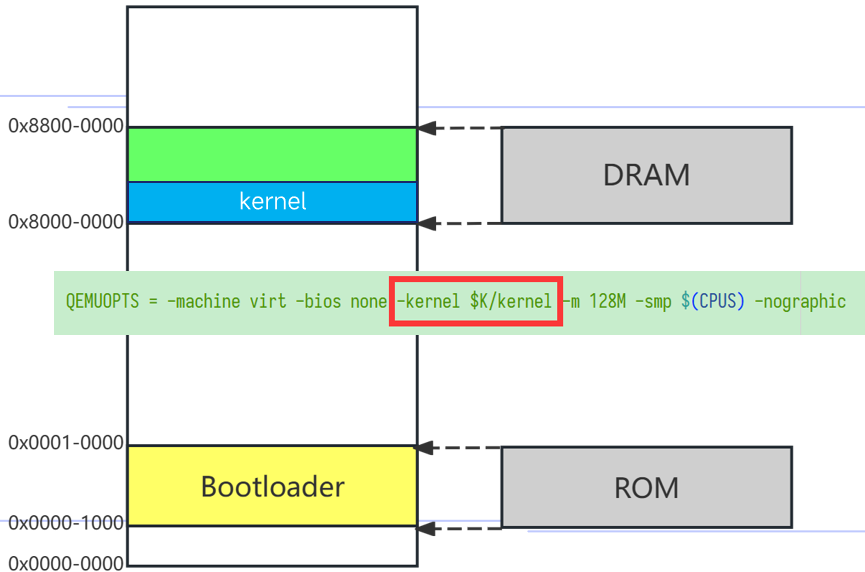
```
[VIRT_RTL] = { 0x101000, 0x1000 },
[VIRT_CLINT] = { 0x200000, 0x1000 },
[VIRT_ACLINT_SSWI] = { 0x2F0000, 0x4000 },
[VIRT_PCIE_PIO] = { 0x300000, 0x1000 },
[VIRT_IOMMU_SYS] = { 0x301000, 0x1000 },
[VIRT_PLATFORM_BUS] = { 0x400000, 0x2000000 },
[VIRT_PLIC] = { 0xc00000, VIRT_PLIC_SIZE(VIRT_CPUS_MAX * 2) },
[VIRT_APLIC_M] = { 0xc00000, APLIC_SIZE(VIRT_CPUS_MAX) },
[VIRT_APLIC_S] = { 0xd00000, APLIC_SIZE(VIRT_CPUS_MAX) },
[VIRT_UART0] = { 0x1000000, 0x100 },
[VIRT_VIRTIO] = { 0x1000100, 0x1000 },
[VIRT_FW_CFG] = { 0x1010000, 0x18 },
[VIRT_FLASH] = { 0x2000000, 0x4000000 },
[VIRT_IMSIC_M] = { 0x2400000, VIRT_IMSIC_MAX_SIZE },
[VIRT_IMSIC_S] = { 0x2800000, VIRT_IMSIC_MAX_SIZE },
[VIRT_PCIE_ECAM] = { 0x3000000, 0x1000000 },
[VIRT_PCIE_MMIO] = { 0x4000000, 0x4000000 },
[VIRT_DRAM] = { 0x8000000, 0x0 },
};
```



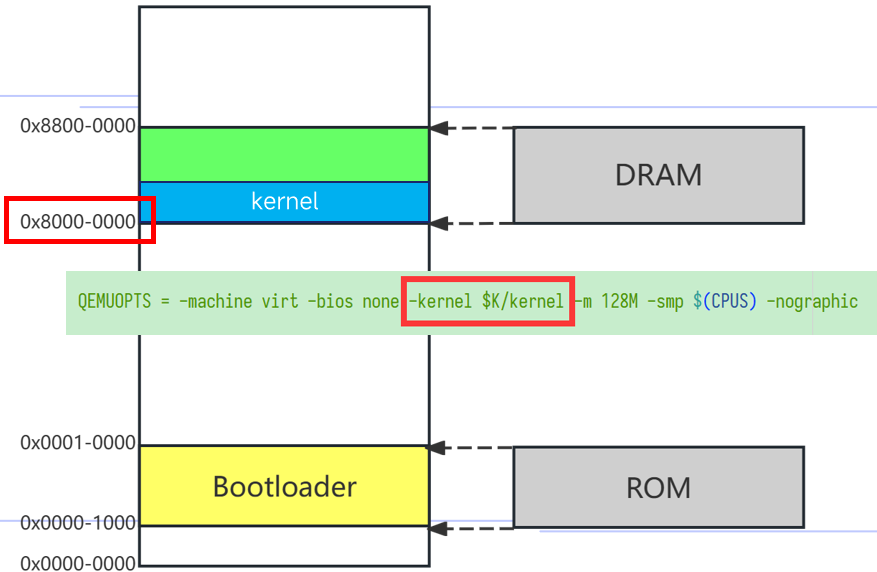
QEMU-virt 的启动过程



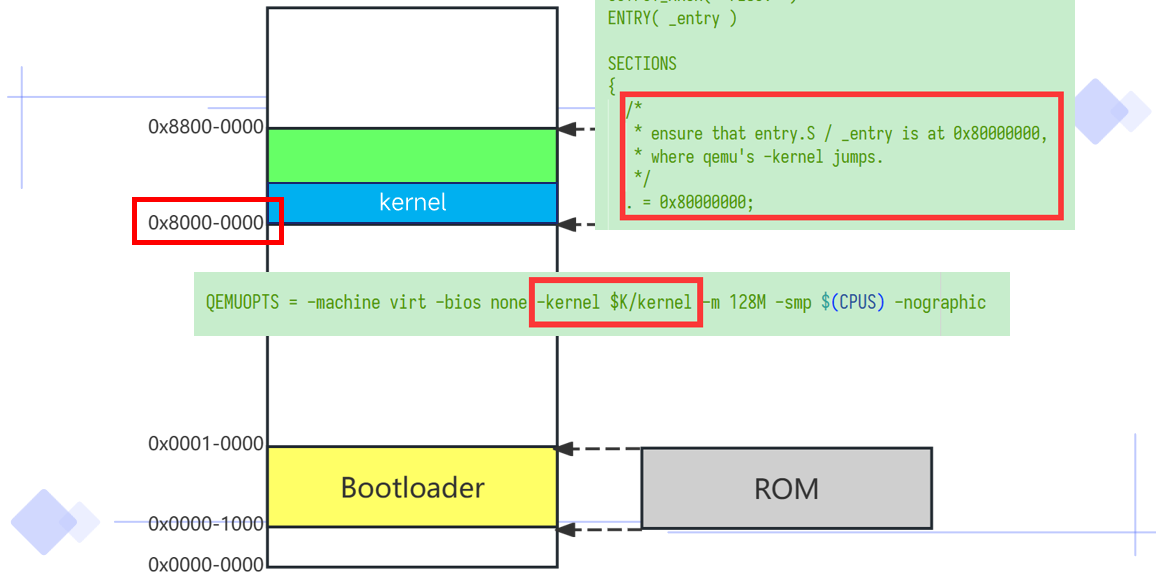
QEMU-virt 的启动过程



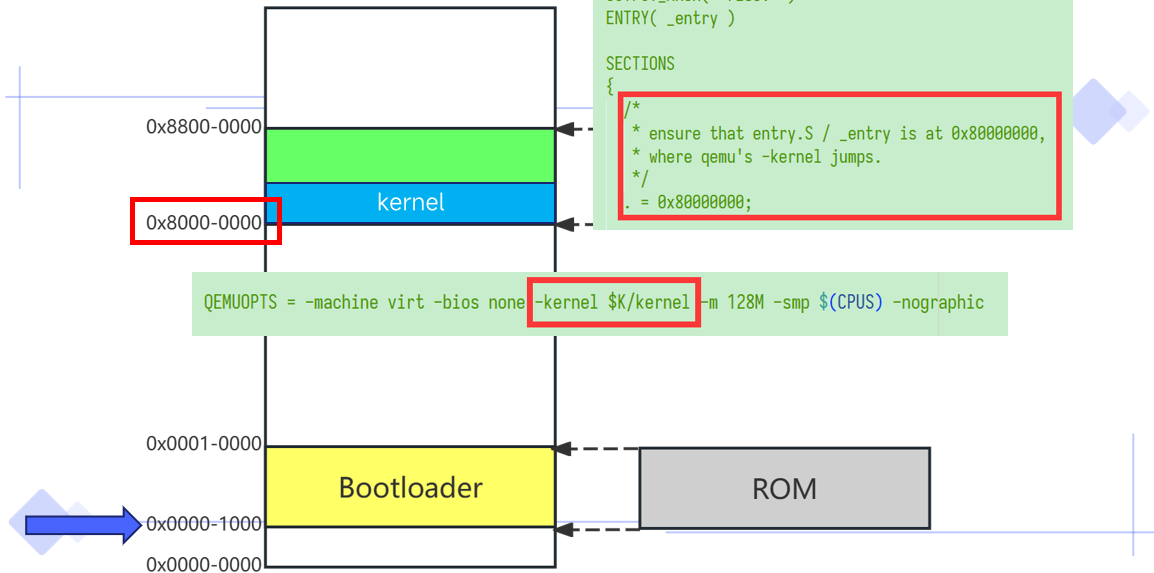
QEMU-virt 的启动过程



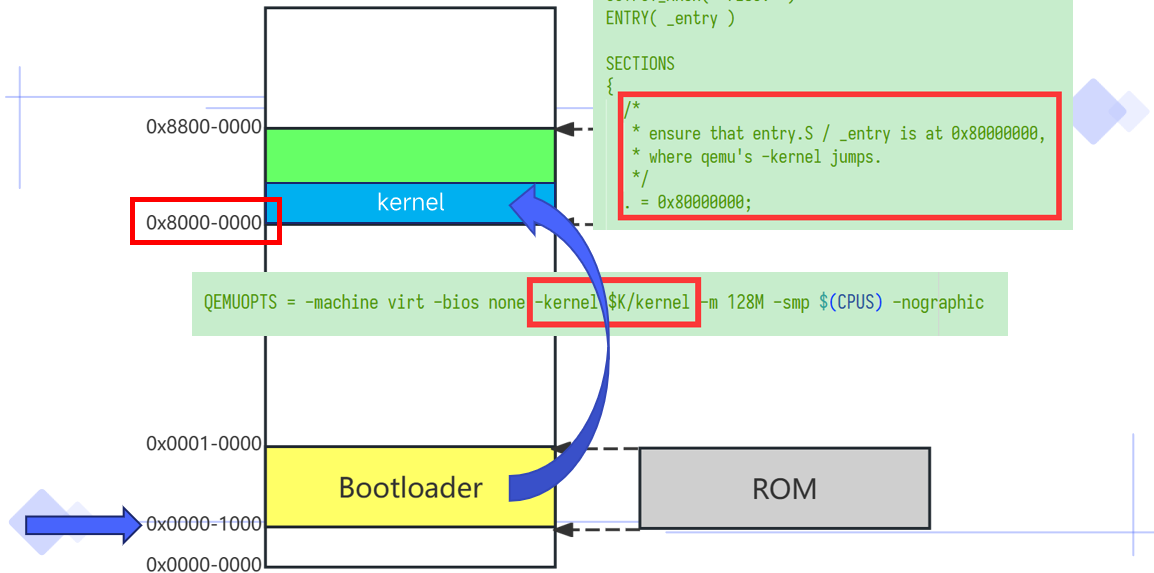
QEMU-virt 的启动过程



QEMU-virt 的启动过程



QEMU-virt 的启动过程





03

xv6 的启动实现

xv6 的启动实现

为 C 语言准备环境



为切换到 S-mode
做准备



切换到 S-mode

xv6 的启动实现

为 C 语言准备环境

为切换到 S-mode
做准备

切换到 S-mode

kernel/entry.S

```
.section .text
.global _entry
_entry:
    # set up a stack for C.
    # stack0 is declared in start.c,
    # with a 4096-byte stack per CPU.
    # sp = stack0 + ((hartid + 1) * 4096)
    la sp, stack0
    li a0, 1024*4
    csrr a1, mhartid
    addi a1, a1, 1
    mul a0, a0, a1
    add sp, sp, a0
    # jump to start() in start.c
    call start

spin:
    j spin
```

xv6 的启动实现

为 C 语言准备环境

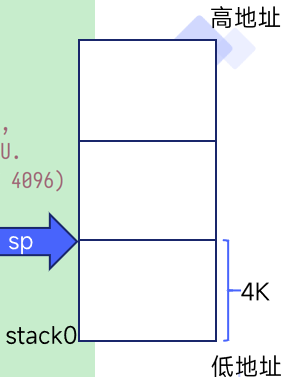
为切换到 S-mode
做准备

切换到 S-mode

kernel/entry.S

```
.section .text
.global _entry
_entry:
    # set up a stack for C.
    # stack0 is declared in start.c,
    # with a 4096-byte stack per CPU.
    # sp = stack0 + ((hartid + 1) * 4096)
    la sp, stack0
    li a0, 1024*4
    csrr a1, mhartid
    addi a1, a1, 1
    mul a0, a0, a1
    add sp, sp, a0
    # jump to start() in start.c
    call start

spin:
    j spin
```



xv6 的启动实现

为 C 语言准备环境



为切换到 S-mode
做准备



切换到 S-mode

Table 1. RISC-V privilege levels.

Level	Encoding	Name	Abbreviation
0	00	User/Application	U
1	01	Supervisor	S
2	10	Reserved	
3	11	Machine	M

Table 2. Supported combination of privilege modes.

Number of levels	Supported Modes	Intended Usage
1	M	Simple embedded systems
2	M, U	Secure embedded systems
3	M, S, U	Systems running Unix-like operating systems

The RISC-V Instruction Set Manual Volume II , Chapter 1.2. Privilege Levels

xv6 的启动实现

为 C 语言准备环境

为切换到 S-mode
做准备

切换到 S-mode

Table 5. Currently allocated RISC-V supervisor-level CSR addresses.

Number	Privilege	Name	Description
Supervisor Trap Setup			
0x100	SRW	sstatus	Supervisor status register.
0x104	SRW	sie	Supervisor interrupt-enable register.
0x105	SRW	stvec	Supervisor trap handler base address.
0x106	SRW	scounteren	Supervisor counter enable.
Supervisor Configuration			
0x10A	SRW	senvcfg	Supervisor environment configuration register.
Supervisor Counter Setup			
0x120	SRW	scountinhibit	Supervisor counter-inhibit register.

Table 7. Currently allocated RISC-V machine-level CSR addresses.

Number	Privilege	Name	Description
Machine Information Registers			
0xF11	MRO	mvendorid	Vendor ID.
0xF12	MRO	marchid	Architecture ID.
0xF13	MRO	mimpid	Implementation ID.
0xF14	MRO	mhartid	Hardware thread ID.
0xF15	MRO	mconfigptr	Pointer to configuration data structure.
Machine Trap Setup			
0x300	MRW	mstatus	Machine status register.
0x301	MDIW	misa	ISA and extensions

xv6 的启动实现

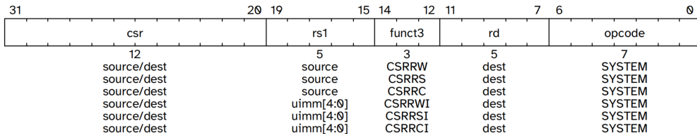
为 C 语言准备环境

为切换到 S-mode
做准备

切换到 S-mode

6.1. CSR Instructions

All CSR instructions atomically read-modify-write a single CSR, whose CSR specifier is encoded in the 12-bit *csr* field of the instruction held in bits 31-20. The immediate forms use a 5-bit zero-extended immediate encoded in the *rs1* field.



The RISC-V Instruction Set Manual Volume I , Chapter 6. "Zicsr"

xv6 的启动实现

为 C 语言准备环境

为切换到 S-mode
做准备

切换到 S-mode

kernel/start.c

```
// entry.S jumps here in machine mode on stack0.
void
start()
{
    // set M Previous Privilege mode to Supervisor, for mret.
    unsigned long x = r_mstatus();
    x &= ~MSTATUS_MPP_MASK;
    x |= MSTATUS_MPP_S;
    w_mstatus(x);

    // set M Exception Program Counter to main, for mret.
    // requires gcc -mcmodel=medany
    w_mepc((uint64)main);

    // disable paging for now.
    w_satp(0);

    // configure Physical Memory Protection to give supervisor mode
    // access to all of physical memory.
    w_pmpaddr0(0x3fffffffffffffffULL);
    w_pmpcfg0(0xf);
}
```

xv6 的启动实现

为 C 语言准备环境

为切换到 S-mode
做准备

切换到 S-mode

kernel/start.c

```
// entry.S jumps here in machine mode on stack0.
void
start()
{
    // set M Previous Privilege mode to Supervisor, for mret.
    unsigned long x = r_mstatus();
    x &= ~MSTATUS_MPP_MASK;
    x |= MSTATUS_MPP_S;
    w_mstatus(x);

    // set M Exception Program Counter
    // requires gcc -mcmodel=medany
    w_mepc((uint64)main);

    // disable paging for now.
    w_satp(0);

    // configure Physical Memory Protection to give supervisor mode
    // access to all of physical memory.
    w_pmpaddr0(0x3fffffffffffffffULL);
    w_pmpcfg0(0xf);
}
```

```
static inline uint64
r_mstatus()
{
    uint64 x;
    asm volatile("csrr %0, mstatus" : "=r" (x) );
    return x;
}
```

xv6 的启动实现

为 C 语言准备环境

为切换到 S-mode
做准备

切换到 S-mode

kernel/start.c

```
// entry.S jumps here in machine mode on stack0.
void
start()
{
    // set M Previous Privilege mode to Supervisor, for mret.
    unsigned long x = r_mstatus();
    x &= ~MSTATUS_MPP_MASK;
    x |= MSTATUS_MPP_S;
    w_mstatus(x);

    // set M Exception Cause to 0
    // requires golden register
    w_mepc(0);

    // disable paging for now.
    w_satp(0);

    // configure Physical Memory Protection to give supervisor mode
    // access to all of physical memory.
    w_pmpaddr0(0xffffffffffffffffull);
    w_pmpcfg0(0xf);
}
```

```
static inline void
w_mstatus(uint64 x)
{
    asm volatile("csrw mstatus, %0" : : "r" (x));
}
```

xv6 的启动实现

为 C 语言准备环境

为切换到 S-mode
做准备

切换到 S-mode

kernel/start.c

```
// entry.S jumps here in machine mode on stack0.
void
start()
{
    // set M Previous Privilege mode to Supervisor, for mret.
    unsigned long x = r_mstatus();
    x &= ~MSTATUS_MPP_MASK;
    x |= MSTATUS_MPP_S;
    w_mstatus(x);
}
```

63	62																	48			
SD	WPRI																				
47	43										42	41	40	39	38	37	36	35	34	33	32
WPRI										MDT	MPPEL	WPRI	MPV	GVA	MBE	SBE	SXL[1:0]		UXL[1:0]		
31											25	24	23	22	21	20	19	18	17	16	
WPRI										SDT	SPELP	TSR	TW	TVM	MXR	SUM	MPRV	XS[1:0]			
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0						
XS[1:0]	FS[1:0]	MPP[1:0]			VS[1:0]	SPP	MPIE	UBE	SPIE	WPRI	MIE	WRI	SIE	WPR							

Figure 8. Machine-mode status (`mstatus`) register for RV64

The RISC-V Instruction Set Manual Volume II, Chapter 3.1.6

```
// configure physical memory protection to give supervisor mode
// access to all of physical memory.
w_pmpaddr0(0x3fffffffffu11);
w_pmpcfg0(0xf);
```

xv6 的启动实现

为 C 语言准备环境

为切换到 S-mode
做准备

切换到 S-mode

kernel/start.c

```
// entry.S jumps here in machine mode on stack0.  
void  
start()  
{  
    // set M Previous Privilege mode to Supervisor, for mret.  
    unsigned long x = r_mstatus();  
    x &= ~MSTATUS_MPP_MASK;  
    x |= MSTATUS_MPP_S;  
    w_mstatus(x);  
}
```

Table 1. RISC-V privilege levels.

Level	Encoding	Name	Abbreviation
0	00	User/Application	U
1	01	Supervisor	S
2	10	Reserved	
3	11	Machine	M

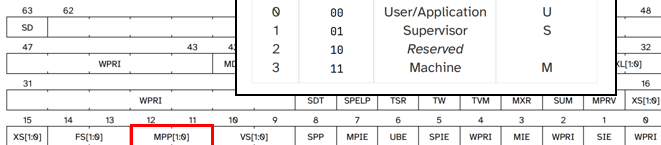


Figure 8. Machine-mode status (mstatus) register for RV64

The RISC-V Instruction Set Manual Volume II, Chapter 3.1.6

```
// configure physical memory protection to give supervisor mode  
// access to all of physical memory.  
w_pmpaddr0(0x3fffffffffffffull);  
w_pmpcfg0(0xf);
```

xv6 的启动实现

为 C 语言准备环境

为切换到 S-mode
做准备

切换到 S-mode

kernel/start.c

```
// entry.S jumps here in machine mode on stack0.
void
start()
{
    // set M Previous Privilege mode to Supervisor, for mret.
    unsigned long x = r_mstatus();
    x &= ~MSTATUS_MPP_MASK;
    x |= MSTATUS_MPP_S;
    w_mstatus(x);

    // set M Exception Program Counter to main, for mret.
    // requires gcc -mcmodel=medany
    w_mepc((uint64)main);
}
```

MXLEN-1

0

mepc

MXLEN

Figure 21. Machine exception program counter register.

The RISC-V Instruction Set Manual Volume II , Chapter 3.1.6

```
w_pmpaddr0(0x3fffffffffffffull);
w_pmpcfg0(0xf);
```

xv6 的启动实现

为 C 语言准备环境

为切换到 S-mode
做准备

切换到 S-mode

kernel/start.c

```
// entry.S jumps here in machine mode on stack0.
void
start()
{
    // set M Previous Privilege mode to Supervisor, for mret.
    unsigned long x = r_mstatus();
    x &= ~MSTATUS_MPP_MASK;
    x |= MSTATUS_MPP_S;
}
```

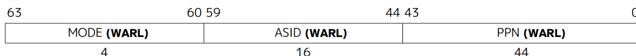


Figure 64. Supervisor address translation and protection (satp) register when SXLEN=64, for MODE values Bare, Sv39, Sv48, and Sv57.

The RISC-V Instruction Set Manual Volume II , Chapter 3.1.6

```
// disable paging for now.
w_satp(0);
```

```
// configure Physical Memory Protection to give supervisor mode
// access to all of physical memory.
w_pmpaddr0(0x3fffffffffffffull);
w_pmpcfg0(0xf);
```

xv6 的启动实现

为 C 语言准备环境

为切换到 S-mode
做准备

切换到 S-mode

kernel/start.c

```
// entry.S jumps here in machine mode on stack0.
void
start()
{
    // set M Previous Privilege mode to Supervisor, for mret.
    unsigned long x = r_mstatus();
    x &= ~MSTATUS_MPP_MASK;
    x |= MSTATUS_MPP_S;
    w_mstatus(x);

    // set M Exception Program Counter to main, for mret.
    // requires gcc -mcmodel=medany
    w_mepc((uint64)main);

    // disable paging for now.
    w_satp(0);

    // configure Physical Memory Protection to give supervisor mode
    // access to all of physical memory.
    w_pmpaddr0(0x3fffffffffffffffull);
    w_pmpcfg0(0xf);
}
```

xv6 的启动实现

为 C 语言准备环境

为切换到 S-mode
做准备

切换到 S-mode

kernel/start.c

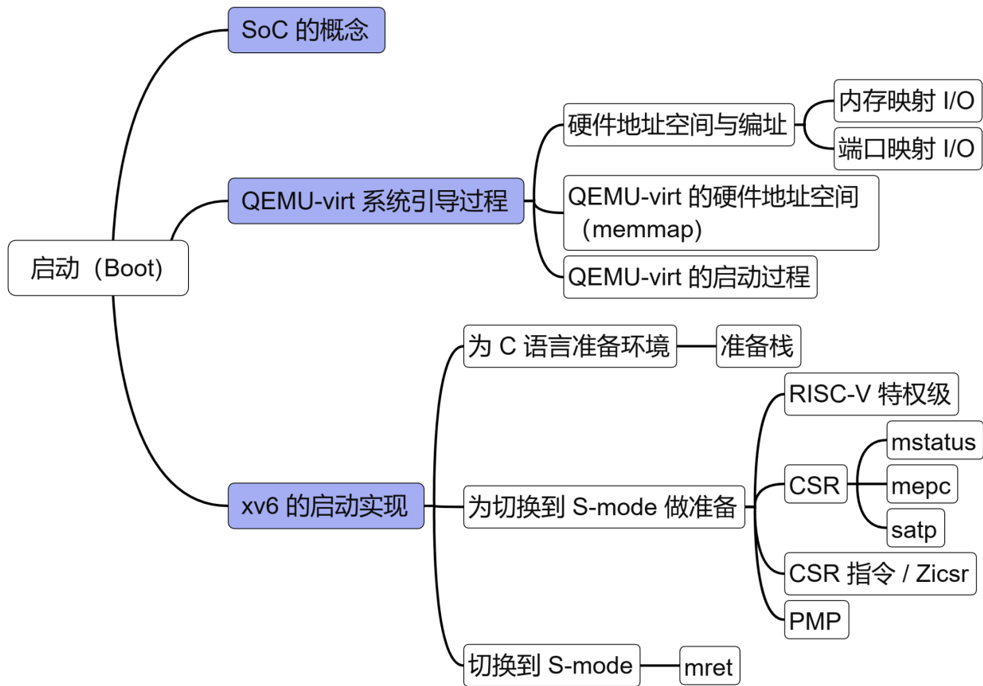
```
// entry.S jumps here in machine mode on stack0.  
void  
start()  
{  
    // 省略 .....  
  
    // switch to supervisor mode and jump to main().  
    asm volatile("mret");  
}
```

kernel/main.c

```
// start() jumps here in supervisor mode on all CPUs.  
void  
main()  
{  
    while(1);  
}
```



本章总结



谢谢

